Resolução de Problemas

JOÃO LUÍS TAVARES DA SILVA

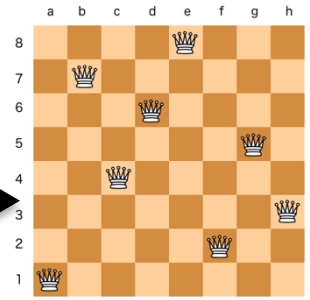
Problemas

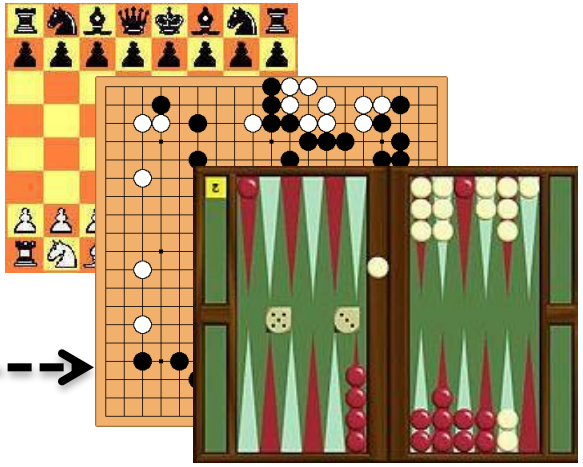
- Problemas estão relacionados à resolução de tarefas humanas:
  - aquisição de informações(sentidos);
  - processamento de informações (raciocínio); e,
  - ações no meio exterior (capacidade de agir).

- Formalmente, um problema constitui-se em uma coleção de informações (conhecimento, regras, objetivos) que serão usadas para decidir o que fazer (solução).

Toy Problems

- **Jogos de tabuleiros**
  - Jogo das *n* rainhas
  - damas, xadrez, go, gamão, etc.

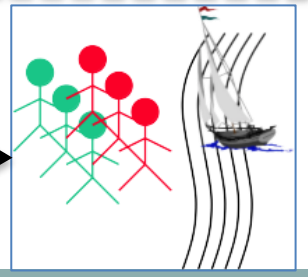




- **Jogos combinatórios**
  - Jogo dos *n* números (*n-puzzle* ou *taquin*)
  - Criptoaritmética
  - Palavras cruzadas
  - Canibais e missionários

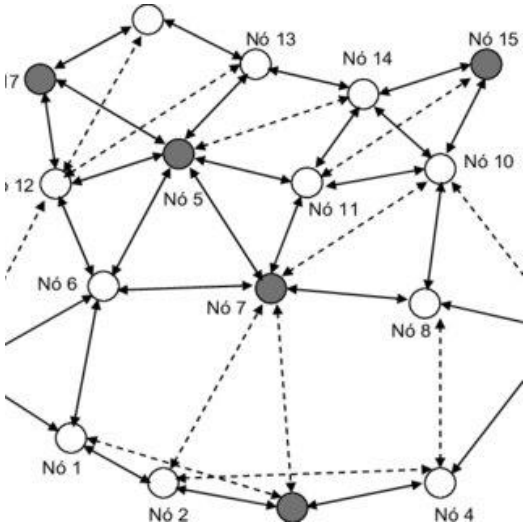
4	5	8
	1	6
7	2	3

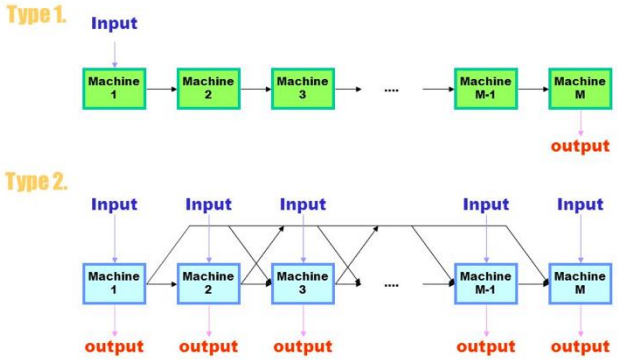
```
send
+more
-----
money
```

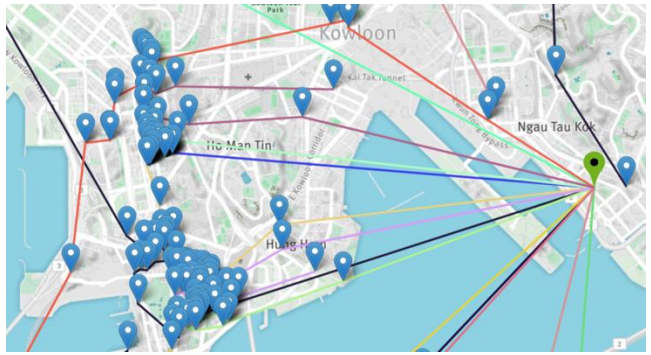


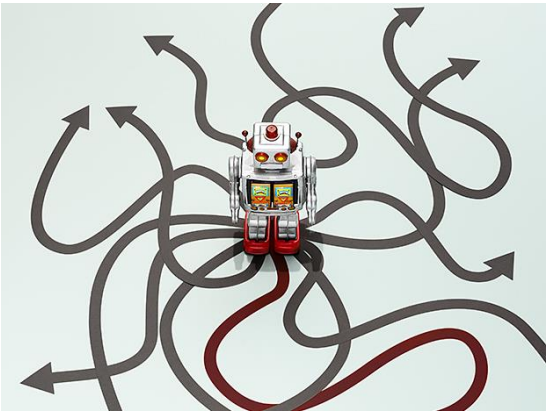
Problemas Reais

- **Cálculo de rotas**
  - rotas em redes de computadores
  - sistemas de planejamento de viagens
  - planejamento de rotas de aviões
  - Caixeiro viajante
- **Alocação (*Scheduling*)**
  - Salas de aula
  - Máquinas industriais (*job shop*)
- **Projeto de *VLSI***
  - *Cell layout*
  - *Channel routing*
- **Navegação de Robôs**
- **Sequência de montagem**







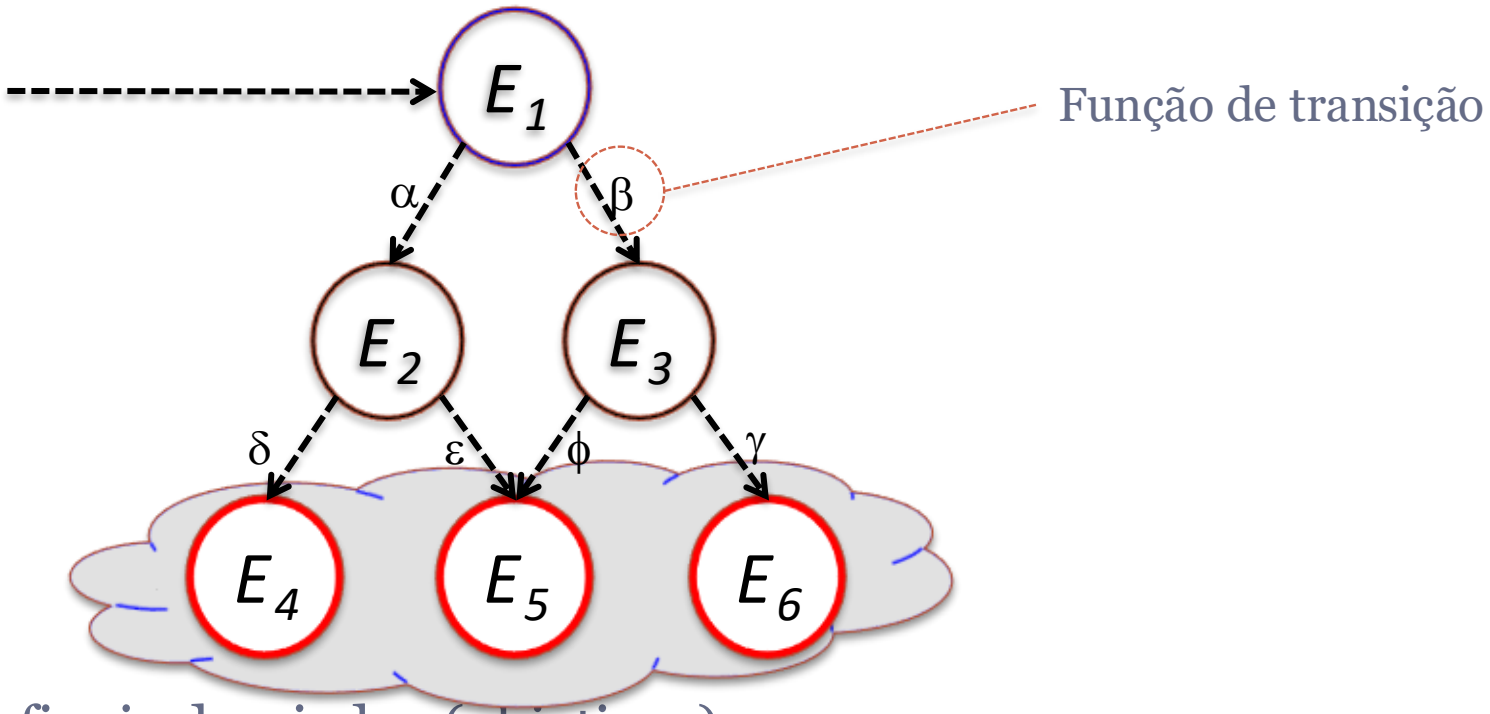


O que é um Problema?

- Em IA, um problema é definido em termos de:
  - **um espaço de estados possíveis**
  - **um conjunto de ações**
  - **um teste de objetivo**
  - **funções de custo**

O que é um Problema?

- **Espaço de estados**
  - um estado inicial;



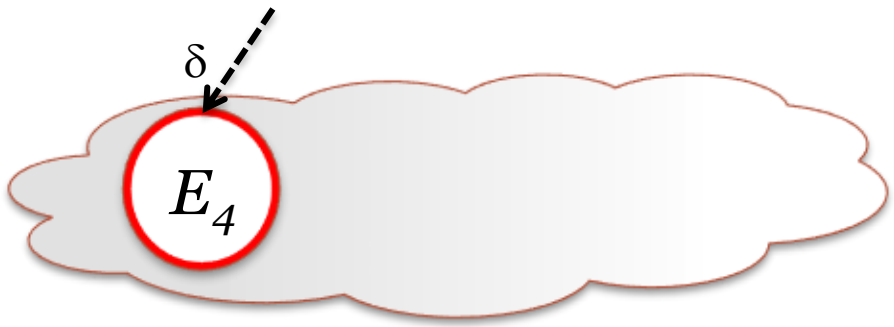
  - um conjunto de estados finais desejados (objetivos)

O que é um Problema?

- **Conjunto de ações:**
  - relacionado ao espaço de estados, define como se movimentar no espaço;
  - Símbolos de ação: *move(x,y)*, *abre(z), produz(x)*, *remove*, *insere*, etc.
  - Função de transição de estados:
    - $\delta(E_1, move(E_1,E_2))=E_2$ → Estando no estado E_1, a ação $move(E_1,E_2)$ muda para o estado E_2.

O que é um Problema?

- **Teste de objetivo:**
  - determina se o estado atingido pertence ao conjunto de estados objetivos.
  - Se o estado atual do espaço é E_4 e $E_4 \in OBJETIVOS$:



O que é um Problema?

- **Função de custo:**
  - indica a **qualidade** do caminho descoberto no espaço.
  - Quantidade de movimentos;
  - Distância entre estado inicial e objetivo;
  - Recursos consumidos;
  - Tempo de percurso;
  - Etc...

O que é resolver um Problema?

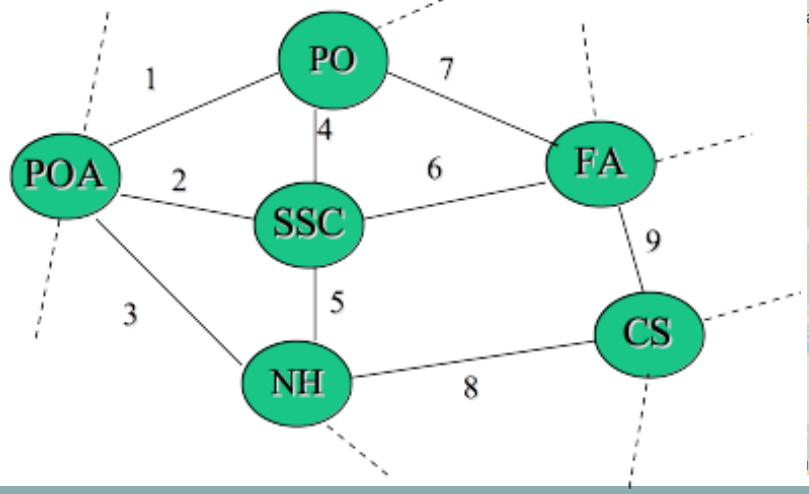
- **Solução:**
  - Sequência de **ações** que levam de um **estado inicial** a um **estado objetivo** desejado.
- **Solução ótima:**
  - Solução com a menor **função de custo**.
- **Etapas na resolução de um problema:**
  - Formulação do Problema;
  - Análise do Problema formalizado;
  - Identificação do conhecimento necessário;
  - Seleção de um técnica de resolução.

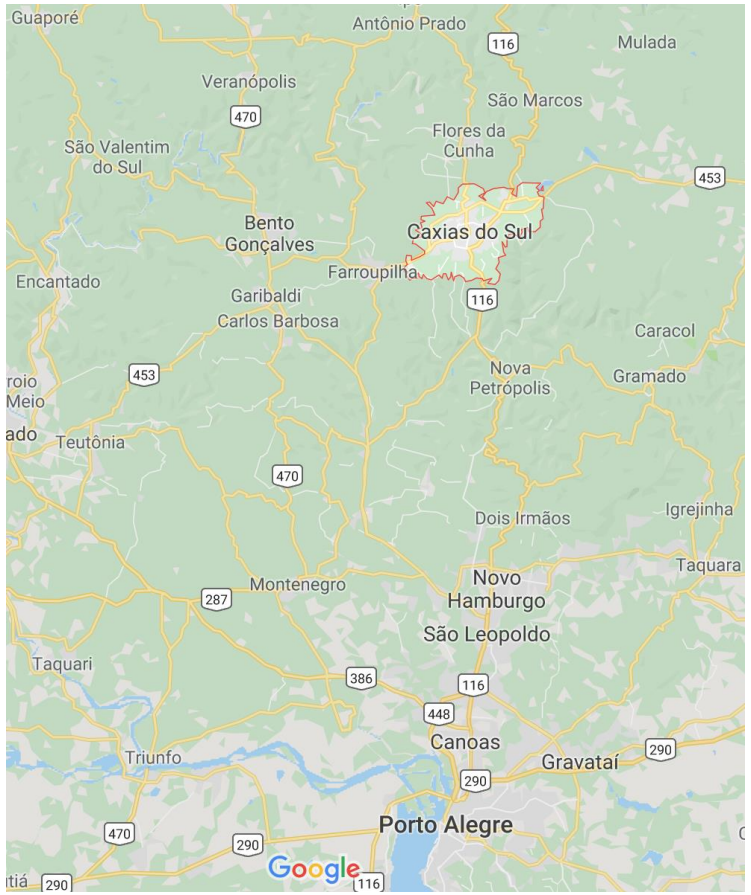
Como **Formular** um Problema?

- O primeiro passo para solucionar um problema é:
  - **Criar uma descrição formal manipulável para o problema.**
- Esta descrição deve conter:
  - a definição de um **espaço de estados** possíveis;
  - a especificação dos **estados iniciais** ;
  - a especificação de um conjunto de estados que sejam aceitáveis como soluções (**estados objetivos**);
  - a especificação do conjunto de **ações** disponíveis (regras ou operadores);

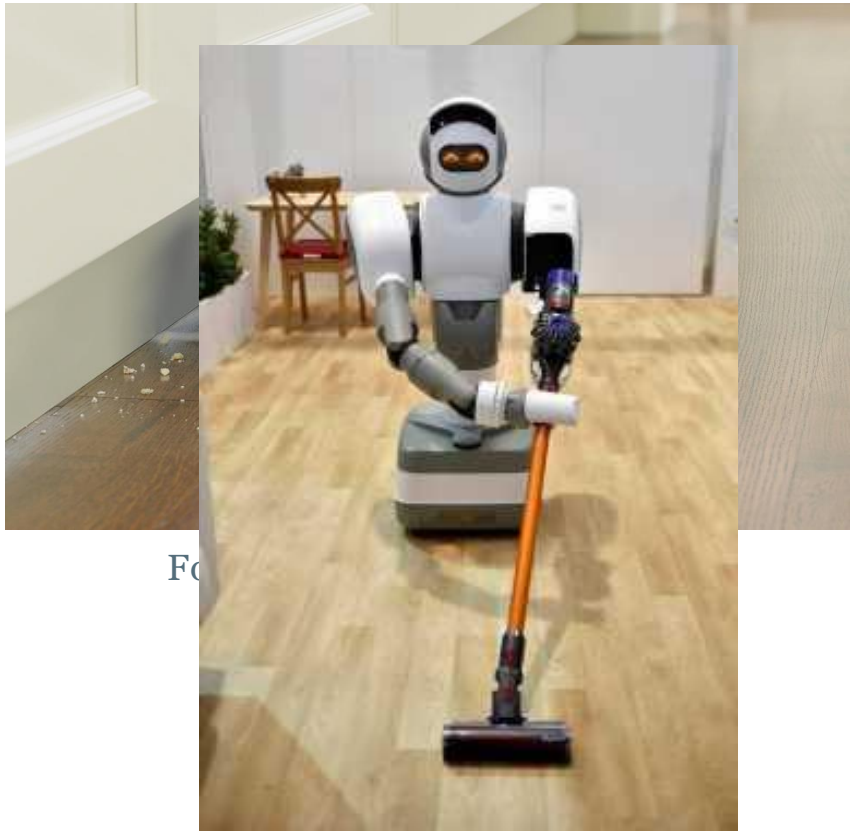
Grafo de busca (de POA à CS)

- Ir de POA à CS:
  - **Estados** = cada possível cidade do mapa;
  - **Estado inicial** = POA;
  - **Teste de término** = estar em CS;
  - **Operadores** = dirigir de uma cidade para outra;
  - **Custo do caminho** = número de cidades visitadas, distância percorrida, tempo de viagem, grau de divertimento, etc.



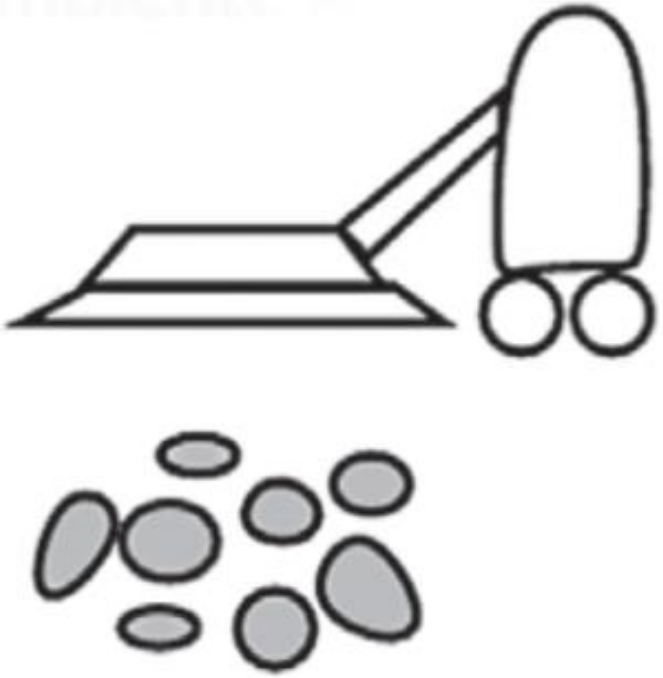


Exemplo Robô Aspirador



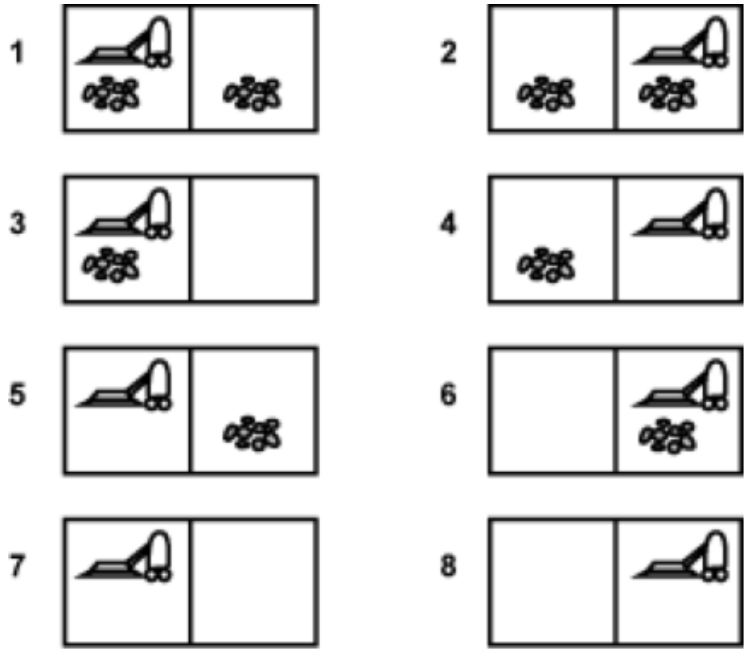
Fo

Fonte: https://aeolusbot.com



Formulação do robô aspirador

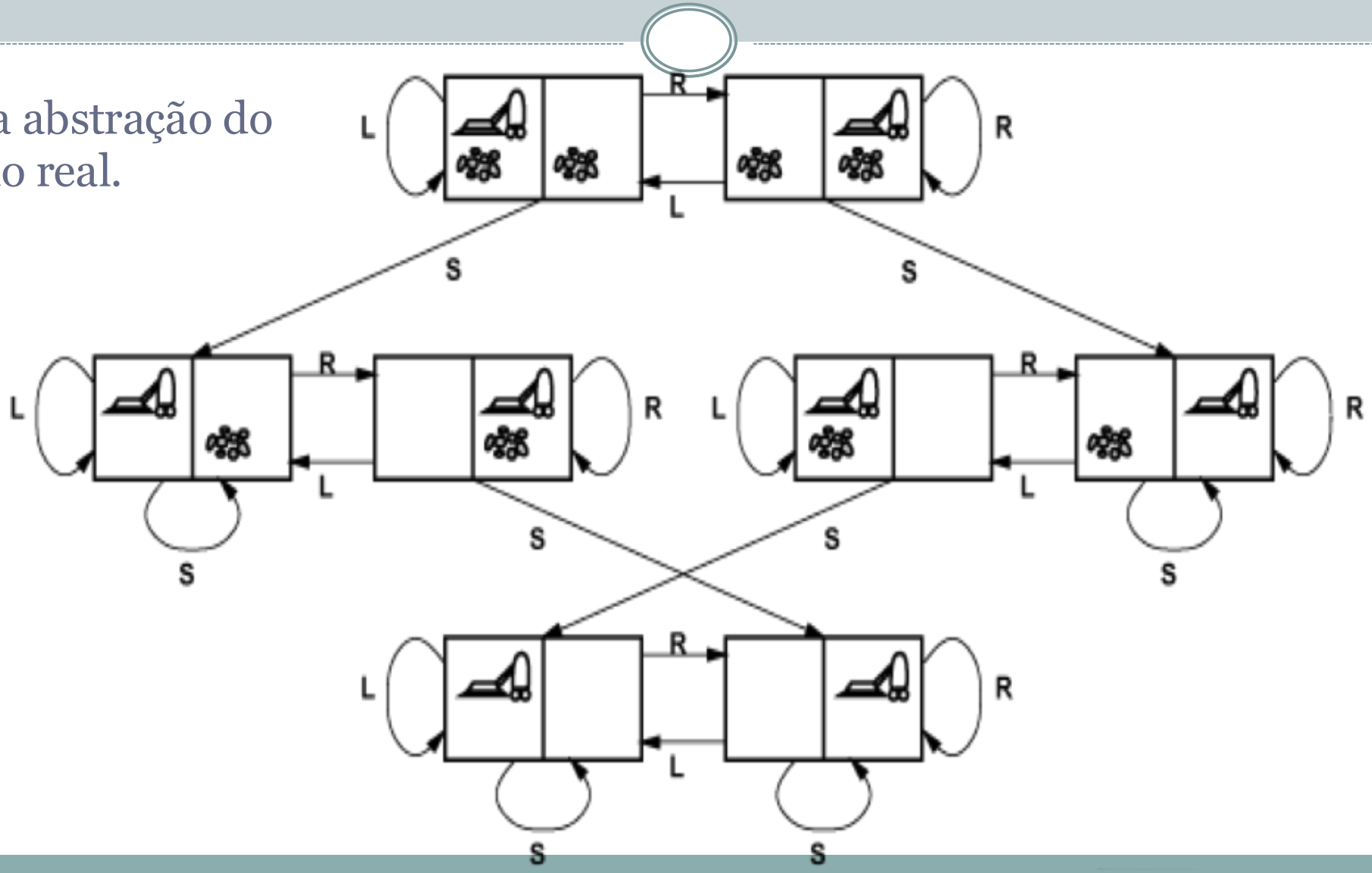
Espaço de Estados possíveis: ----->



- **Estado inicial:** qualquer um dos estados possíveis;
- **Estados finais:** estados nº *7* ou ***8***;
- **Teste de objetivo**: as duas salas limpas;
- **Ações:** mover direita (***R***), mover esquerda (***L***), aspirar (***S***);
- **Custo do caminho:** 1 unidade a cada passo;

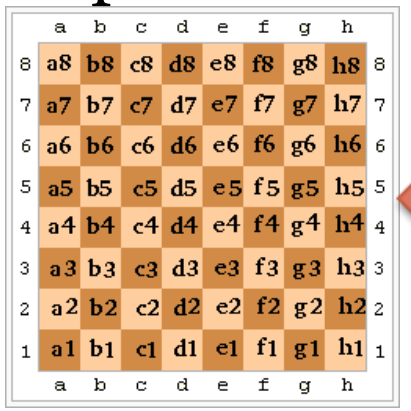
Espaço de estados

- É uma abstração do mundo real.



Formulação Jogo de Xadrez

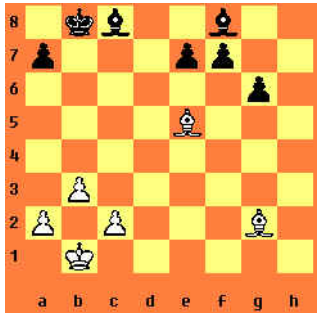
- **Espaço de estados:** matriz 8x8, letras e números representam as peças e posições;
- **Estado inicial:** posição oficial de abertura do Xadrez:



	a	b	c	d	e	f	g	h	
8	1	0	1	1	0	1	1	1	8
7	0	0	1	1	1	1	1	1	7
6	0	1	0	0	1	0	0	0	6
5	1	0	0	0	0	0	0	0	5
4	1	0	1	0	0	1	0	1	4
3	1	0	0	1	1	0	1	0	3
2	0	1	0	1	1	0	0	0	2
1	0	1	1	1	0	0	1	1	1
	a	b	c	d	e	f	g	h	

Bitboard

- **Ações:** regras de movimentação do xadrez:
  - Exemplo, notação algébrica: 1. d4 d5 2. c4 c6 3. cxd5 cxd5 4. Cc3 Cc6 5. Cf3 Cf6;
- **Estados finais:** qualquer estado onde o rei adversário é atacado sem outros movimentos válidos;



Como **Analisar** um Problema?

- **Formulação do problema e do objetivo:**
  - quais são os estados e as ações a considerar?
  - qual é (e como representar) o objetivo?
- **Solução do problema:**
  - processo que gera sequências de ações para alcançar um objetivo;
  - custo do caminho = qualidade da solução.
- **Execução:**
  - executar a solução completa encontrada, ou
  - intercalar execução com busca: planejamento.

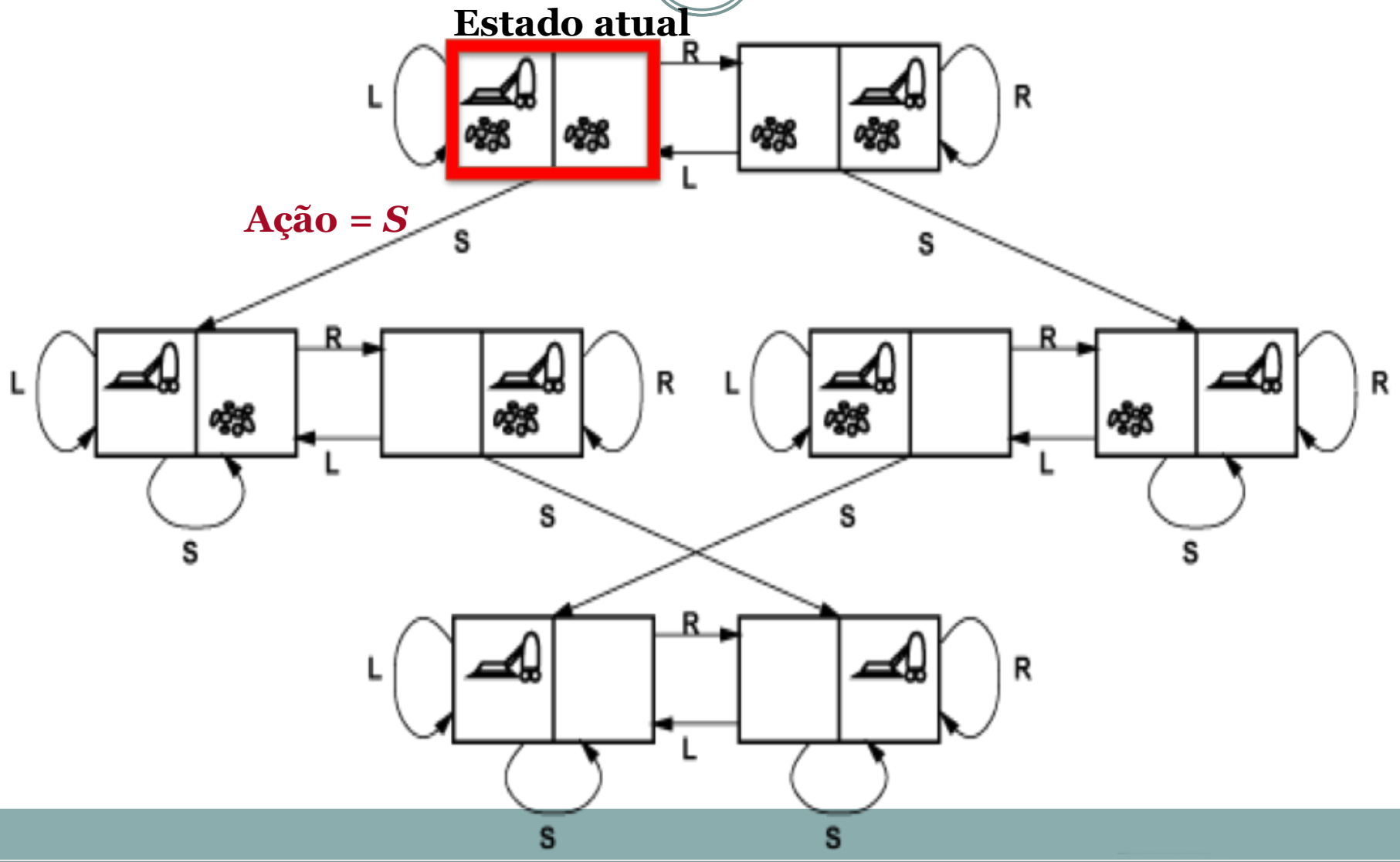
Como **Analisar** um Problema?

- **Formulação do problema e do objetivo:**
  - Lista de estados de 1 à 8 no formato [*(x,y)*], onde *x* e *y* podem assumir um dos dois valores: "*sujo*" e "*limpo*"
  - Elemento do espaço de estados no formato [*(x,y)*], onde *x* = *y* = "*limpo*"

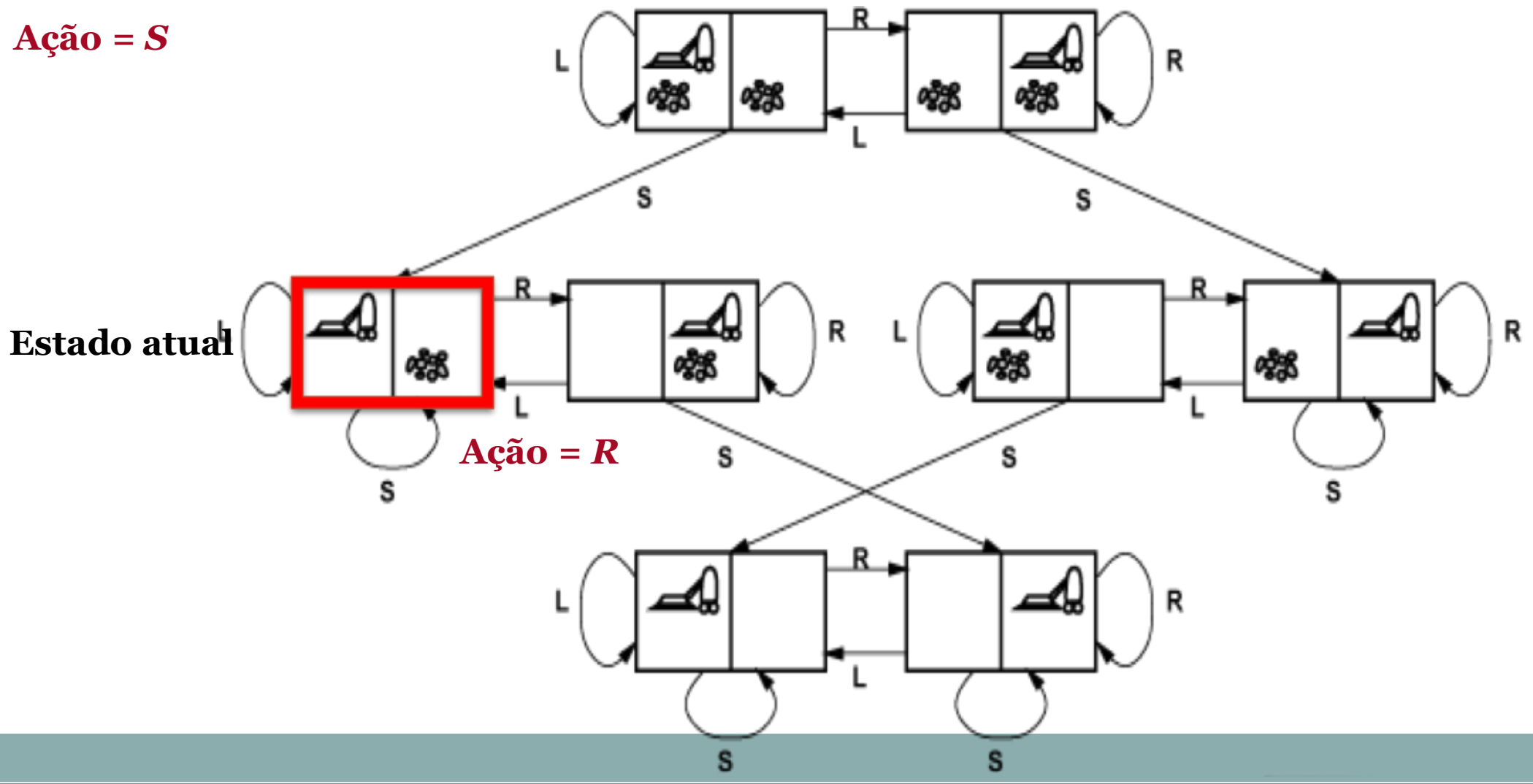
- **Solução do problema:**
  - solução = caminho entre o estado inicial e um estado final;

- **Execução:**
  - Fazer uma busca no espaço de estados, anotando o caminho

Espaço de estados

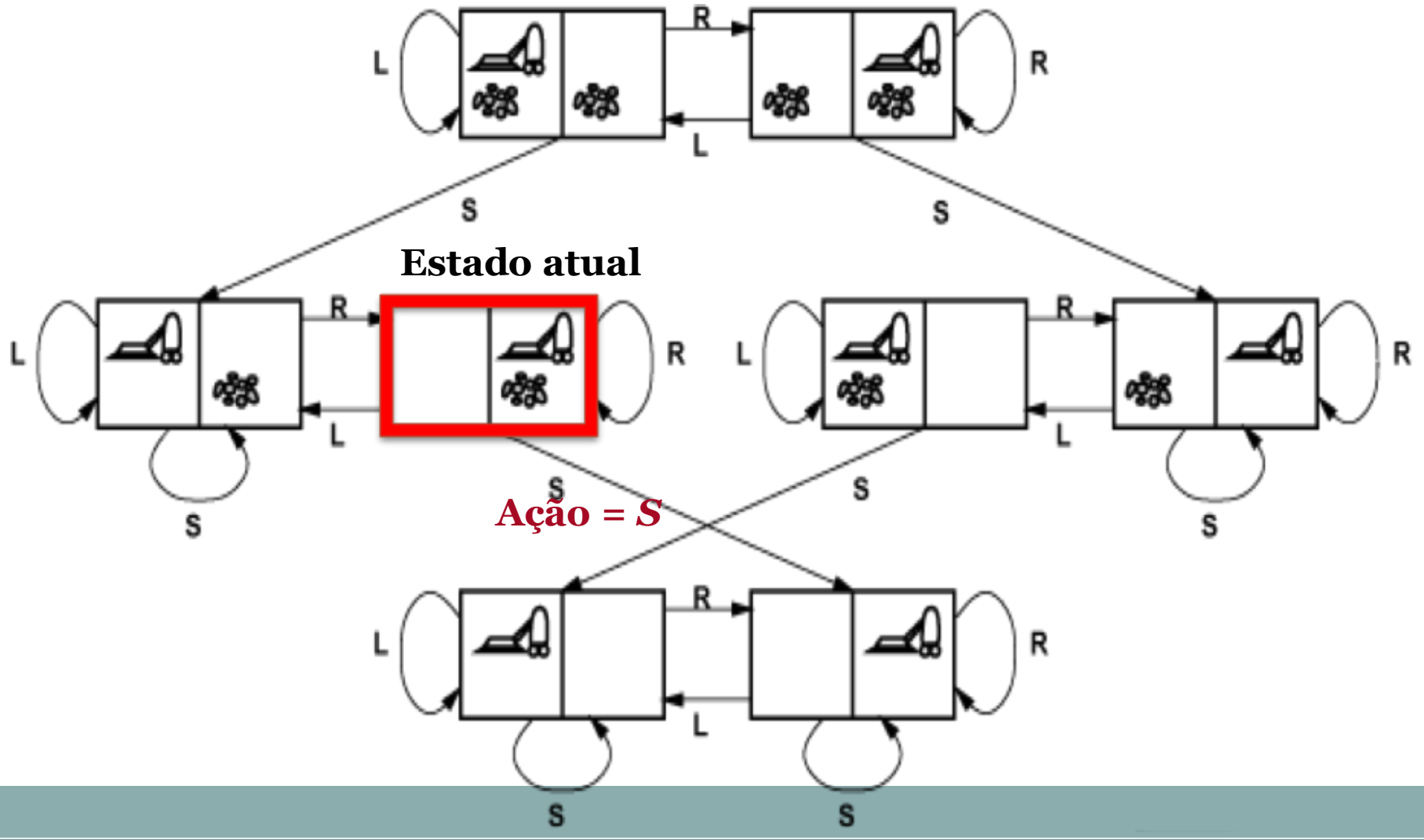


Espaço de estados



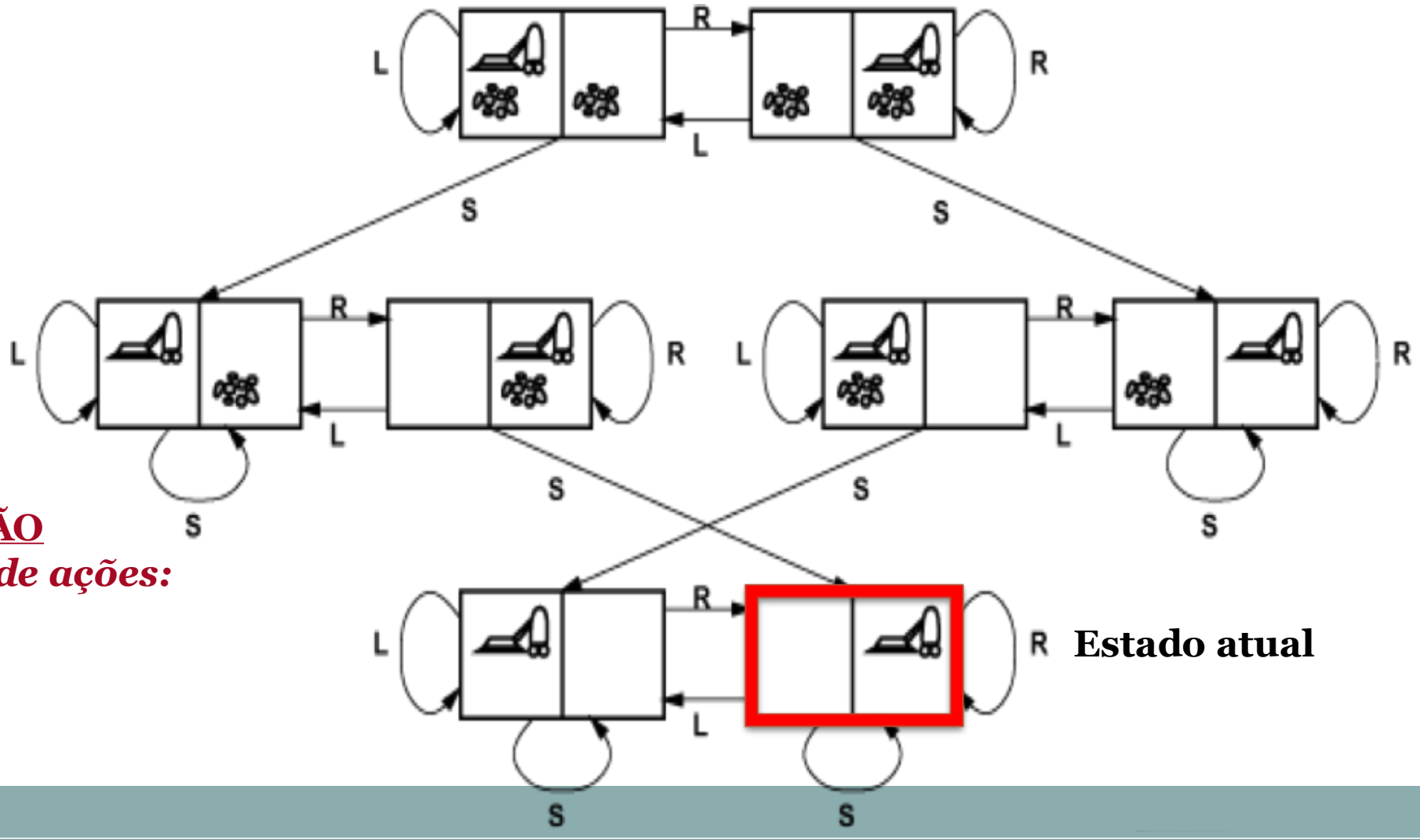
Espaço de estados

Ação = *S*
Ação = *R*



Espaço de estados

Ação = *S*
Ação = *R*
Ação = *S*



SOLUÇÃO

Sequência de ações:
{S, R, S}

Técnicas de **Resolução**

- **Busca**:
  - método de resolução de um problema sem abordagem direta, mas com uma estrutura disponível: Exemplo: **Busca em grafos**
- **Utilização do conhecimento**:
  - método de resolução de problemas complexos explorando a estrutura dos objetos envolvidos. Exemplo: **Planejamento**
- **Abstração**:
  - método que separa características e variações importantes de outras irrelevantes no processo de resolução. Exemplo: **Aprendizagem**

Tipos de Problemas

Definidos em função do conhecimento do sistema:

- Problemas de **estado único** (*Single-state problem*)
- Problemas de **múltiplos estados** (*Multiple-state problem*)
- Problemas de **contingência** (*Contingence problem*)
- Problemas **exploratórios** (*Exploratory problem*)

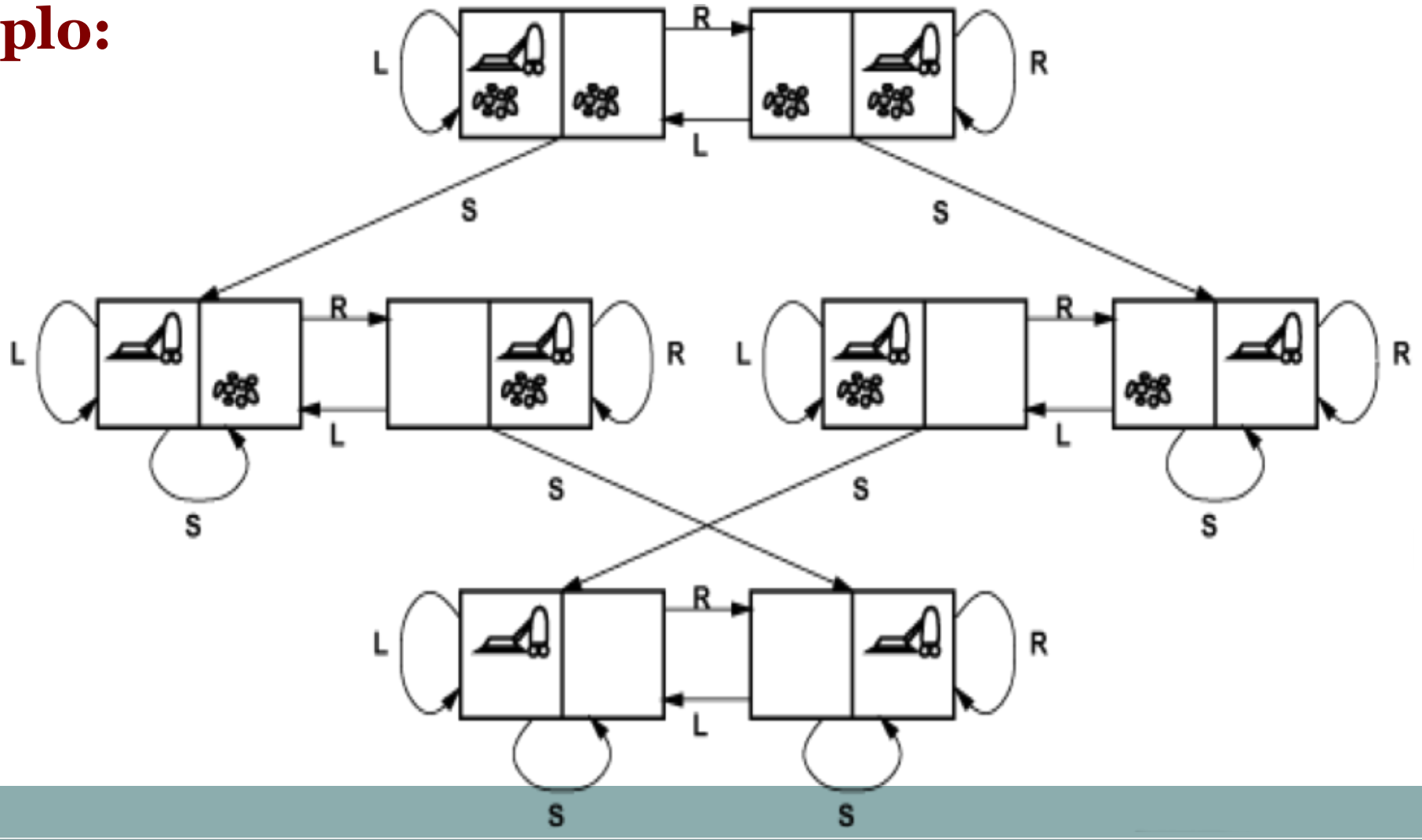
- Definidos em função do tipo de **Ambiente**, tipo de **Conhecimento** envolvido e percepção sobre as **Ações**.

Problemas de estado único

- **Mundo** é *observável*, *determinístico*, *estático*, *discreto*;
- **Conhecimento**
  - Sistema sabe em que estado está (mundo totalmente acessível)
  - Sistema conhece o efeito de cada uma de suas ações
- **Cada ação leva a um único estado**
  - ex. Se o aspirador de pó está no estado 5:
    - A sequência de ações = [direita, aspirar] leva ao estado 8 (final)
- Técnica a aplicar: **BUSCA (*Searching*)**

Problemas de estado único

- **Exemplo:**

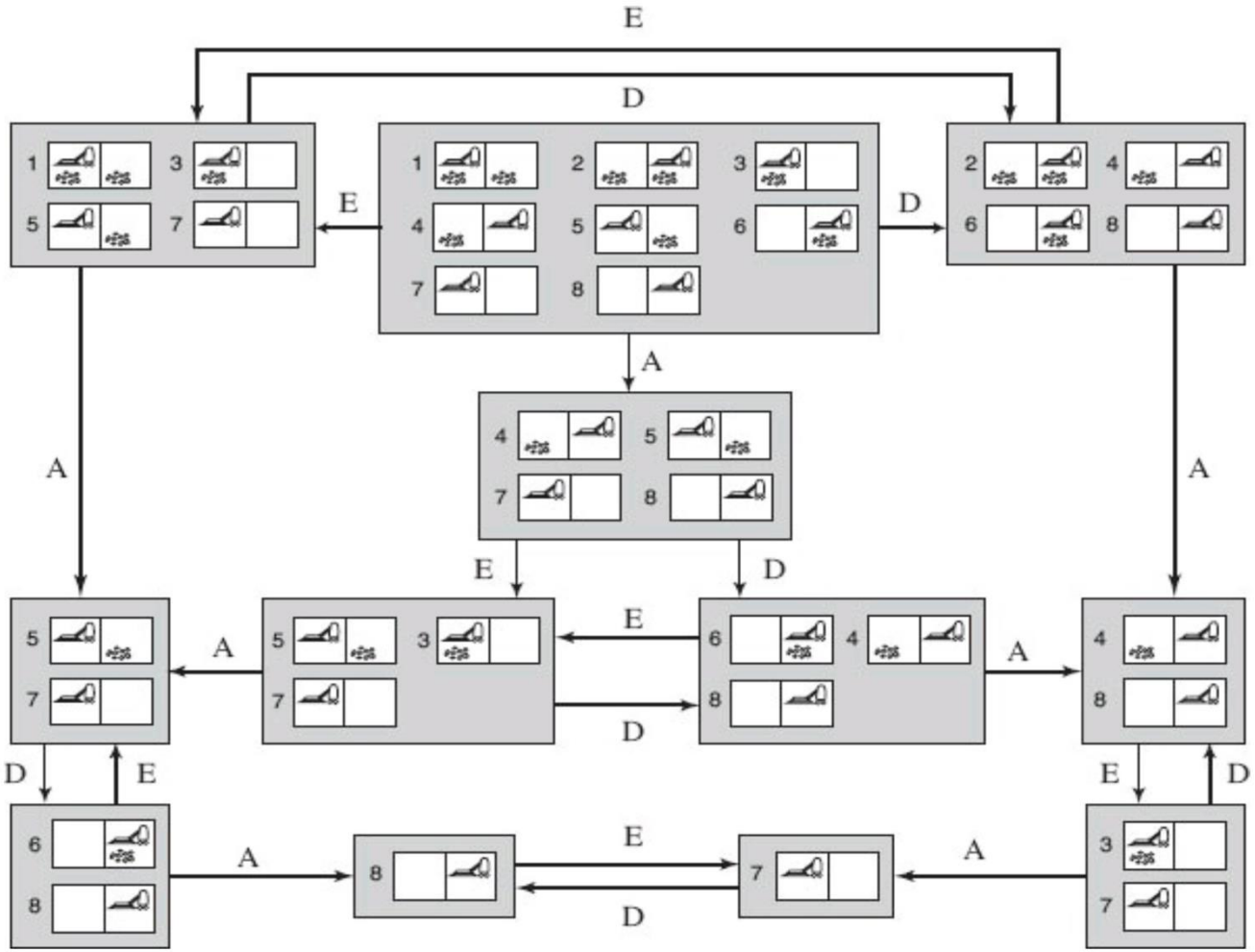


Problemas de múltiplos estados

- **Mundo** *parcialmente observável*, *determinístico*, *estático*, *discreto*;
- **Conhecimento**
  - conhece o efeito de suas ações, mas não o seu estado inicial (percepção deficiente), OU
  - Conhece seu estado inicial, mas não o efeito das ações (execução deficiente);
- **Uma ação pode levar a um conjunto de estados**
- Técnica a aplicar: **BUSCA (*Searching*)**

Problemas de múltiplos estados

- **Exemplo:**



Problemas de contingência

- **Mundo** *parcialmente observável*, *não-determinístico*, *dinâmico*;
- **Conhecimento**
  - não conhece o ambiente inteiro ou não sabe precisar o efeito das ações.
  - Exemplo: Robô enxerga apenas o quarto onde está, precisa usar um mapa
- **Não há sequência prévia de ações que garanta a solução do problema**
  - É necessário intercalar busca e execução:
  - Exemplo: o robô só pode decidir aspirar quando chegar ao quarto
- Técnica a aplicar: **PLANEJAMENTO(*Planning*)**
  - Uma **árvore de ações,** onde cada ramo lida com uma **contingência.**

Problemas exploratórios

- **Conhecimento**:
  - Desconhecimento dos possíveis estados E
  - Desconhecimento sobre o efeito de suas ações
    - ex. estar perdido em uma cidade desconhecida sem mapa.

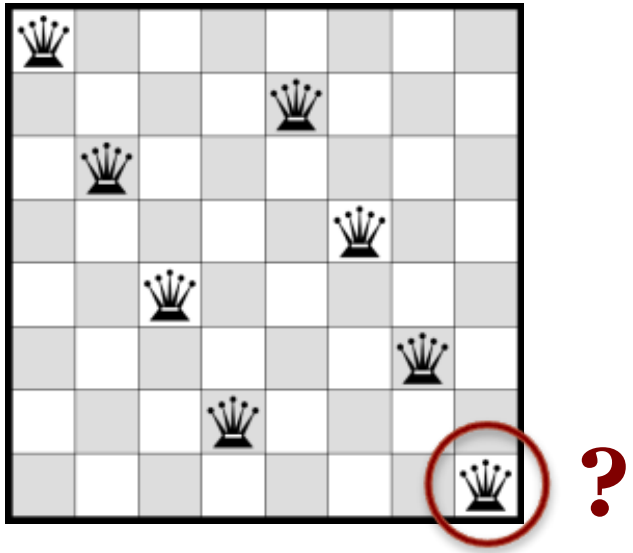
- É necessário **explorar o ambiente**, descobrindo gradualmente o resultado das possíveis ações e os estados existentes.

- Se o sistema "sobreviver", terá aprendido um mapa do ambiente, que poderá ser reutilizado em problemas subsequentes.

- Técnica a aplicar: **APRENDIZAGEM(*Learning*)**

Importância da **Formulação**

- Exemplo: Jogo das 8 Rainhas
  - dispor 8 rainhas no tabuleiro de forma que não possam se “atacar”
    - não pode haver mais de uma rainha em uma mesma linha, coluna ou diagonal
  - somente o custo da busca conta
    - não existe custo de caminho

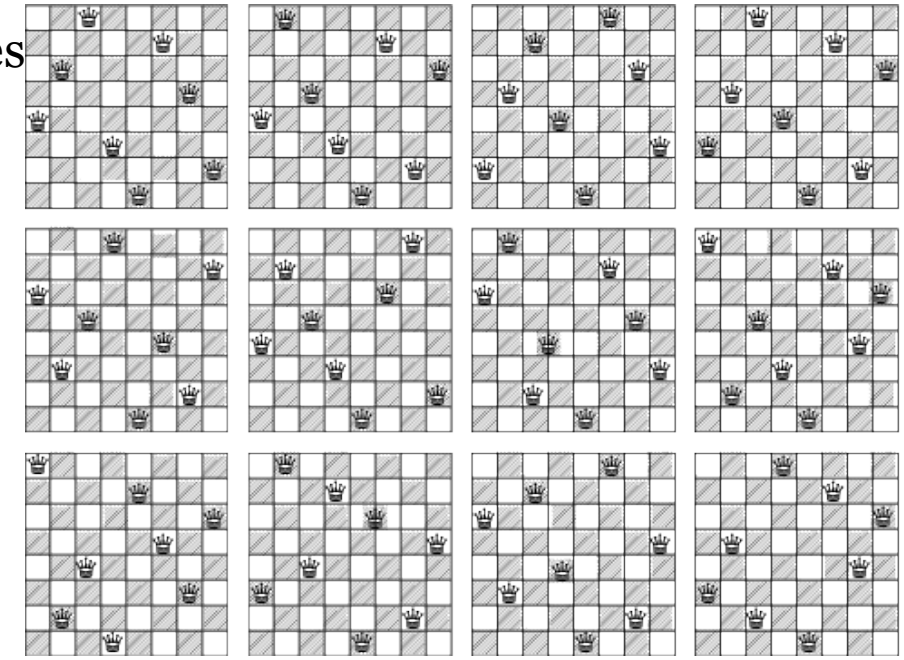


- Diferentes estados e operadores possíveis
  - escolha pode ter consequências boas ou ruins na complexidade da busca ou no tamanho do espaço de estados

N-queens – Quantidade de Soluções

- Soluções Totais:
  - Todas as soluções possíveis
- Soluções Únicas:
  - sem soluções simétricas (rotação e reflexão)

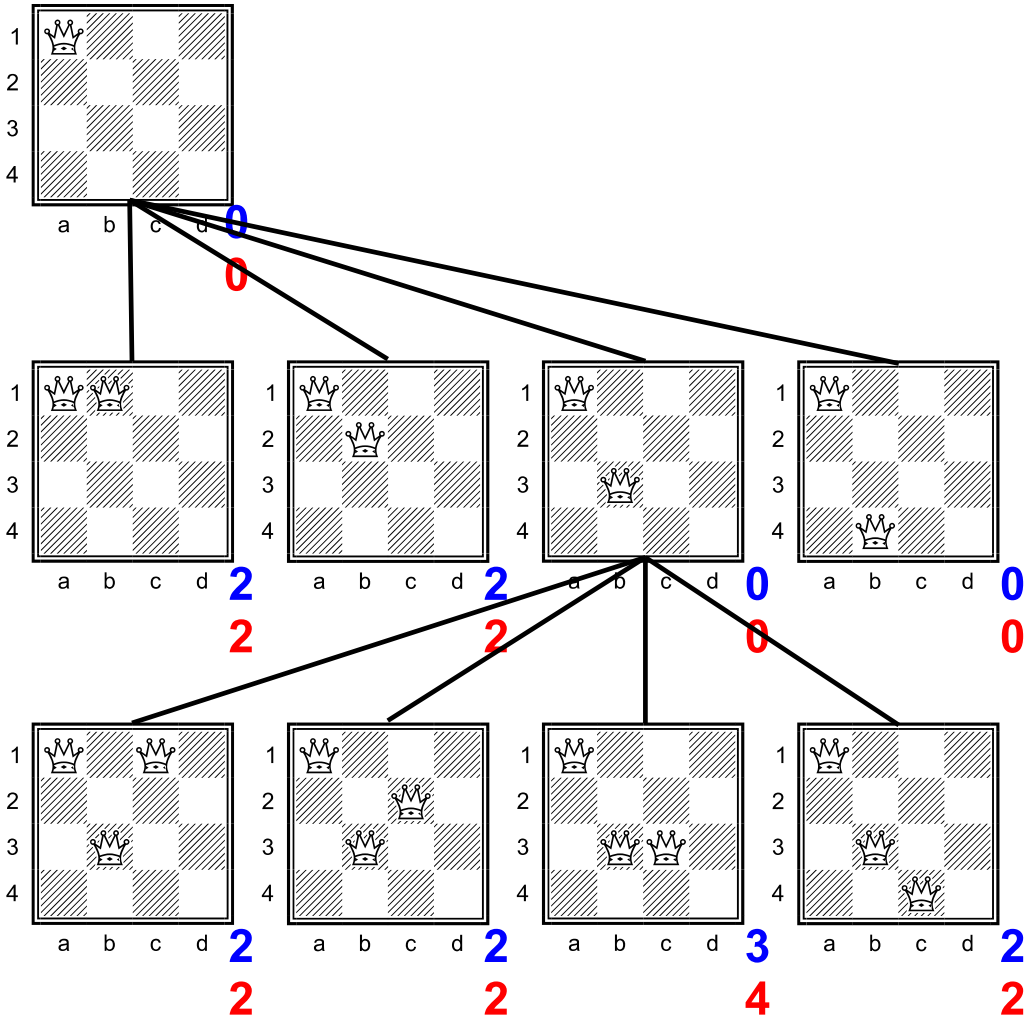
12 soluções
8-queens



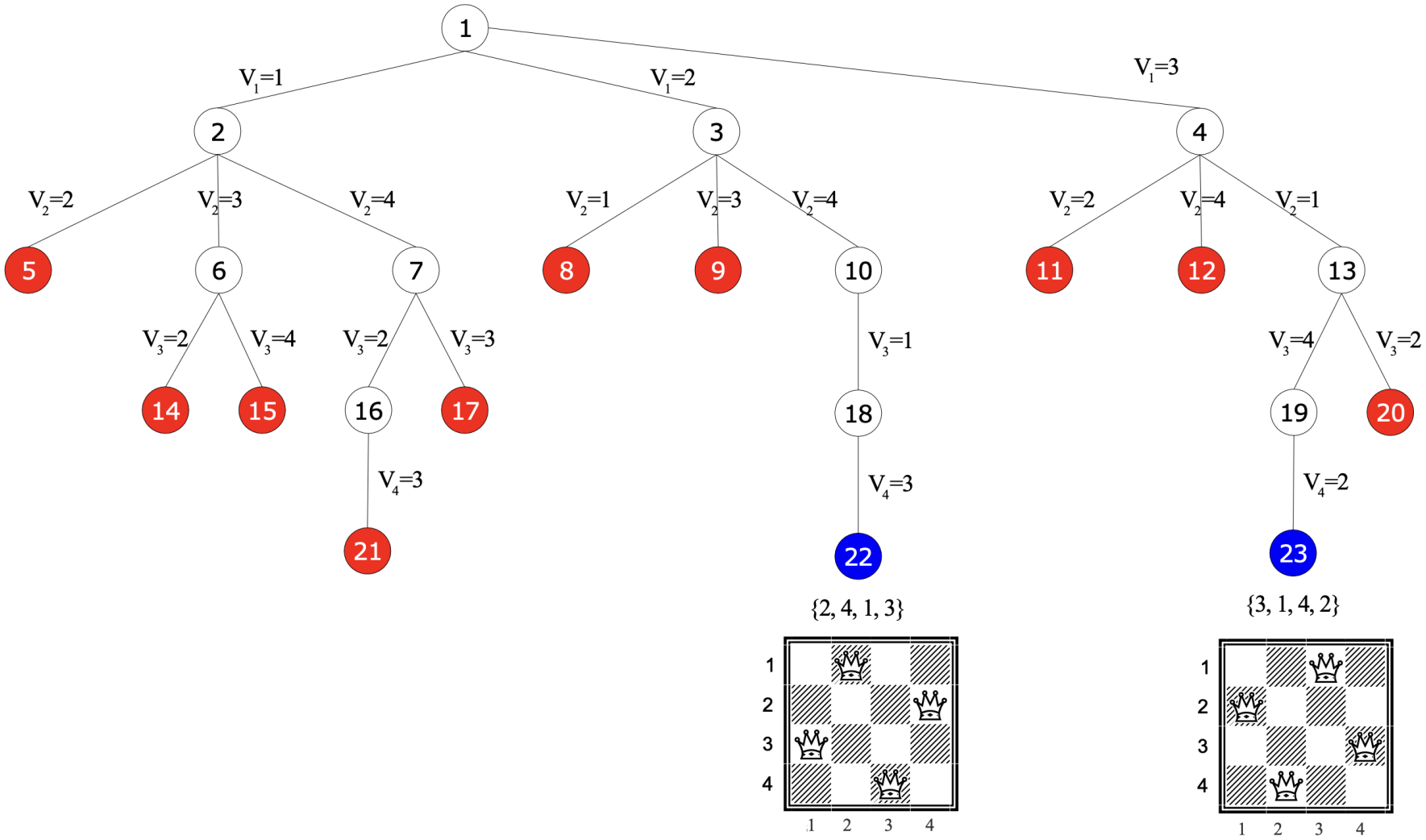
Rainhas	Soluções Totais	Soluções Únicas
1	1	1
2	0	0
3	0	0
4	2	1
5	10	2
6	4	1
7	40	6
8	92	12
9	352	46
10	724	92
...	...	...
26	$2{,}23 \times 10^{16}$	$2{,}78 \times 10^{15}$

N-Queens – Formulação A

- **Estados**: qualquer disposição com *n* rainhas
- **Operadores**: adicionar uma rainha a qualquer quadrado
- **Efetividade**:
  - *Uma combinação simples de* $\binom{64}{8}$ *possibilidades*
  - *Apenas 4.426.165.368 possibilidades*

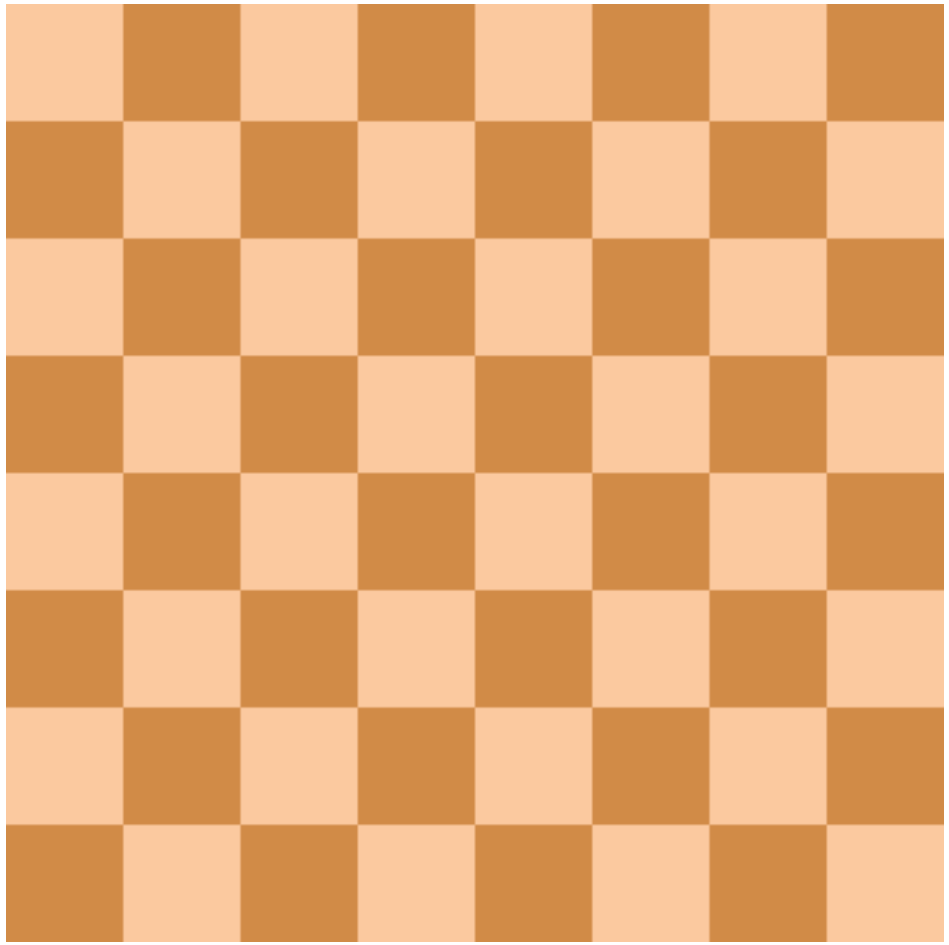


N-Queens – Formulação A



8-Queens – Formulação B

- **Estados**: qualquer disposição com *n=8* rainhas sem ataque mútuo = teste gradual
- **Operadores**: adicionar uma rainha por linha e coluna em uma posição sem ataque
- **Efetividade**:
  - *2057 possibilidades*

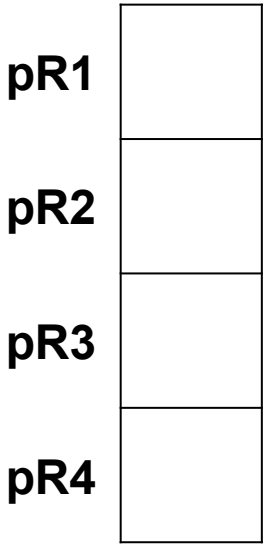


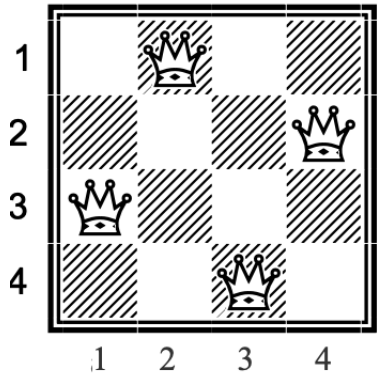
n-Queens – Formulação C

- **Estados**: vetor com n posições $V = <v_1, v_2, ..., v_n>$

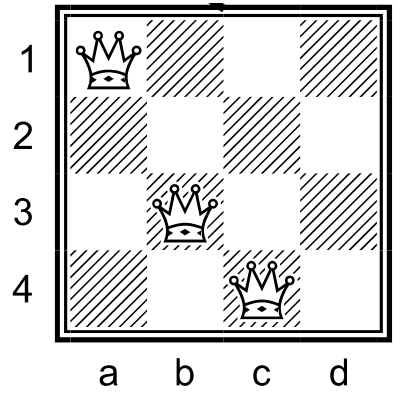
- **Operadores**: adicionar uma rainha na posição v_i.
- **Condições:**
  - nenhum v_i pode ser igual a outro, ou seja, para cada v_i e v_j, $v_i \neq v_j$ (colunas iguais);
  - Para cada v_i e v_j, $i+1 \neq j$ e $v_i+1 \neq v_j$ (mesma diagonal);
  - para cada v_i e v_j, $v_i \neq i$ e $v_j \neq j$ (mesma diagonal principal);
  - para cada v_i e v_j, $i + v_i \neq j + v_j$ (mesma diagonal secundária);
  - Generalizando:
    - $| i - j | == | v_i - v_j |$ ➔ MESMA DIAGONAL

- **Efetividade**:
  - *1024 testes = $O(n^n * n)$*





$V = <2, 4, 1, 3>$



$V = <1, ?, 3, 4>$